✅ COMPLETE MOCK INTERVIEW Q&A (DSA + CORE CS)

1️⃣ What is Time Complexity and Space Complexity?

✅ Correct Answer

Time complexity measures how the number of operations grows with input size and is expressed using Big-O notation, usually considering the worst case.
Space complexity measures the extra memory used by an algorithm, excluding the input, such as auxiliary variables, data structures, and recursion stack.

Example

- Iterating an array → O(n) time, O(1) space
- Using HashMap → O(n) space

2️⃣ Why does recursion use extra space even without data structures?

✅ Correct Answer

Even if no extra data structures are used, recursion consumes memory because each function call is stored on the call stack along with parameters, local variables, and return address. Therefore, space complexity depends on recursion depth.

Example

- Factorial recursion → O(n) stack space
- Recursive binary search → O(log n) stack space

3️⃣ Why does Merge Sort use O(n) space but Quick Sort uses O(log n) space?

✅ Correct Answer

Merge sort requires O(n) extra space because it uses temporary arrays during the merge step. Quick sort sorts the array in place and only uses space for the recursion stack, which is O(log n) on average.

4️⃣ Why is Quick Sort faster than Merge Sort in practice?

✅ Correct Answer

Although both have O(n log n) time complexity, quicksort is faster in practice because it sorts in place, has better cache locality, fewer memory allocations, and lower constant factors compared to merge sort.

5️⃣ Why is Merge Sort preferred over Quick Sort in some systems?

✅ Correct Answer

Merge sort is preferred when stable sorting is required, when sorting linked lists, for external sorting, or when guaranteed O(n log n) worst-case performance is needed. Quicksort can degrade to $O(n^2)$ in the worst case.

6️⃣ Explain the Merge Intervals problem approach.

✅ Correct Answer

First, sort intervals by start time. Then traverse once and merge overlapping intervals by comparing the current interval's end with the next interval's start. If they overlap, merge; otherwise, store the interval.

Complexity

- Time → O(n log n)
- Space → O(n)

7️⃣ Why is space complexity O(n) in Merge Intervals?

✅ Correct Answer

Because the merged intervals are stored in a separate list, which in the worst case can contain all n intervals when no overlaps exist.

8️⃣ Explain Longest Consecutive Sequence (O(n) approach).

✅ Correct Answer

Store all numbers in a HashSet. Only start counting from a number if its predecessor (n-1) does not exist. This ensures each sequence is counted once, giving O(n) time complexity.

9️⃣ Explain Happy Number with cycle detection.

✅ **Correct Answer**

Repeatedly replace the number with the sum of squares of its digits. If it reaches 1, it is happy. If a number repeats, a cycle exists and the number is not happy. A HashSet is used to detect cycles.

1️⃣0️⃣ Why do we return n == 1 in Happy Number?

✅ **Correct Answer**

The loop stops either when n becomes 1 or when a cycle is detected. Returning n == 1 tells us whether the termination happened due to success or due to a cycle.

1️⃣1️⃣ Explain map.getOrDefault(key, 0) + 1.

✅ **Correct Answer**

getOrDefault returns the current value if the key exists; otherwise it returns 0. Adding 1 increments the count, and put stores the updated value back into the map.

1️⃣2️⃣ Design an LRU Cache.

✅ **Correct Answer**

Use a HashMap for O(1) key lookup and a doubly linked list to track usage order. The most recently used item is kept near the tail, and the least recently used near the head. On every get or put, move the node to the tail. If capacity is exceeded, remove the head node.

1️⃣3️⃣ Why do we need a Doubly Linked List in LRU?

✅ **Correct Answer**

Because nodes must be removed and inserted in O(1) time. A singly linked list cannot remove arbitrary nodes efficiently.

1️⃣4️⃣ Why use dummy head and tail in LRU?

✅ **Correct Answer**

Dummy nodes simplify insert and remove operations by avoiding null checks and edge-case handling.

1️⃣5️⃣ Explain remove(node) in LRU.

✅ **Correct Answer**

It reconnects the previous node directly to the next node, effectively removing the current node in O(1) time using pointer updates.

1️⃣6️⃣ When do you prefer BFS over DFS?

✅ **Correct Answer**

BFS is preferred for shortest path problems in unweighted graphs, while DFS is used for traversal, cycle detection, and topological sorting.

1️⃣7️⃣ Difference between recursion and dynamic programming?

✅ **Correct Answer**

Recursion recomputes overlapping subproblems, while dynamic programming stores results to avoid recomputation, improving performance.

1️⃣8️⃣ How do you optimize space in DP?

✅ **Correct Answer**

By observing that only previous states are needed and replacing DP arrays with variables or rolling arrays.

1️⃣9️⃣ Why does HashMap degrade from O(1) to O(log n) in Java 8+?

✅ **Correct Answer**

When too many collisions occur in a bucket, Java 8 converts the bucket from a linked list into a Red-Black tree, making lookup O(log n) instead of O(n).

2️⃣0️⃣ When can HashMap still degrade badly?

✅ **Correct Answer**

When the hashCode() implementation is poor and causes excessive collisions. Even with treeification, a badly distributed hash function can significantly impact performance.
